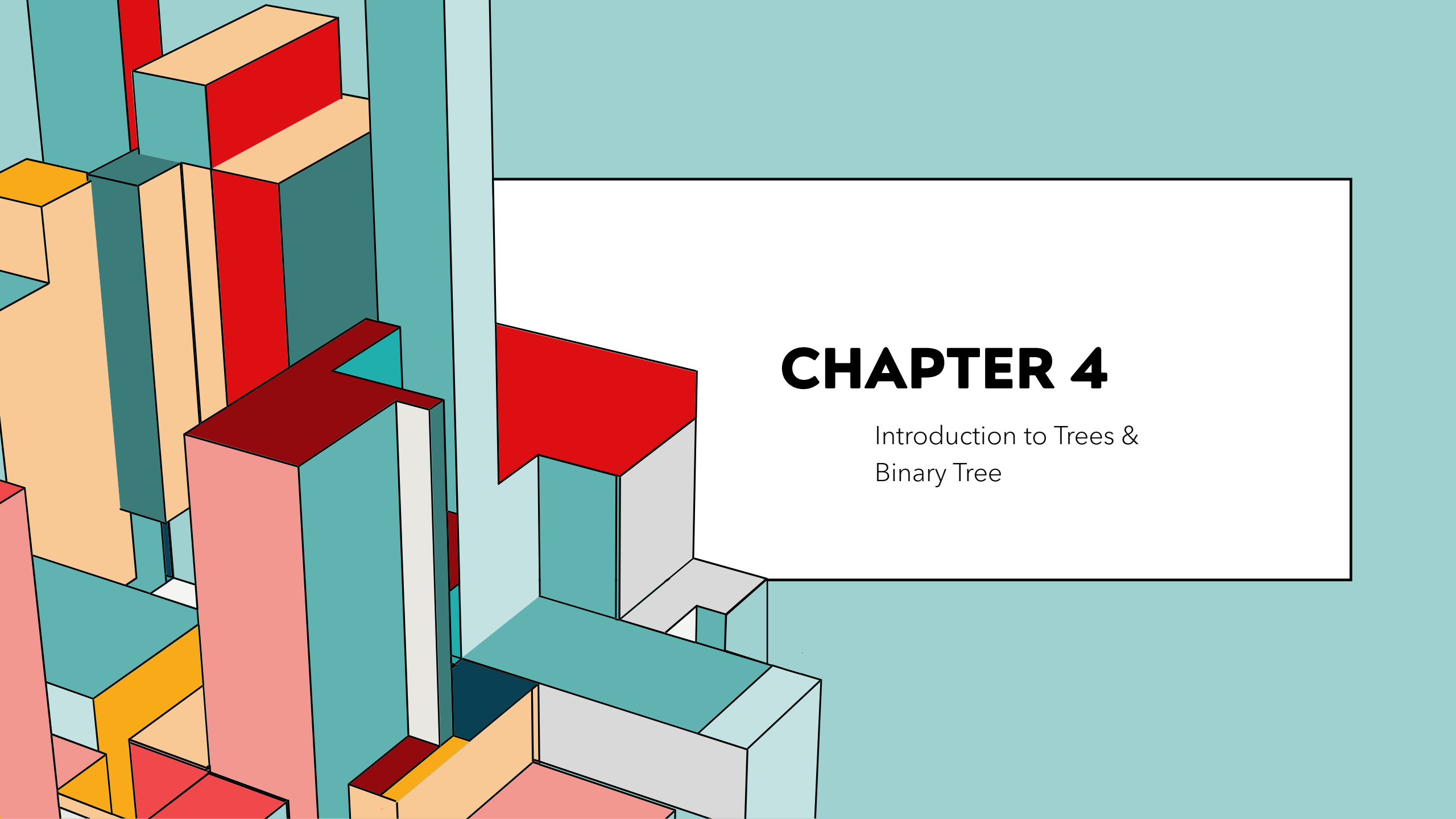




CHAPTER 4

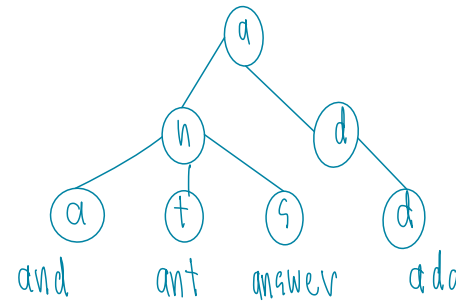
Introduction to Trees &
Binary Tree

Vocab : a

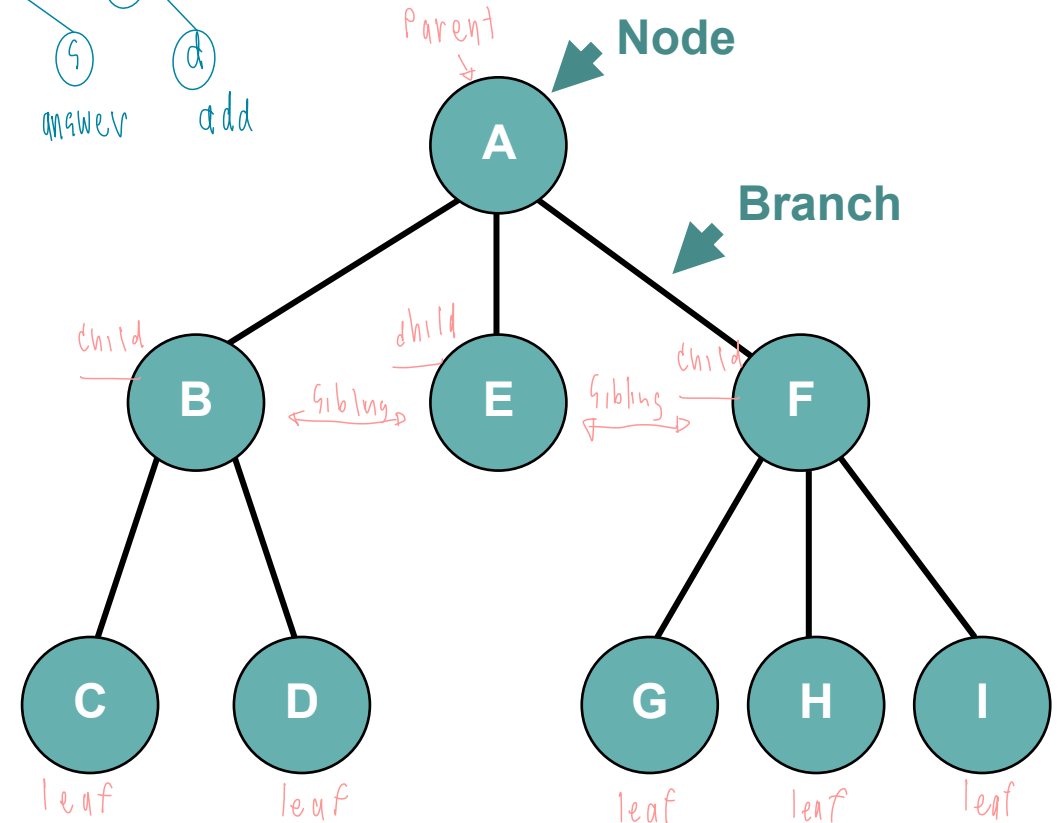
TREES

- เป็นโครงสร้างข้อมูล ที่ประกอบด้วย
 - โหนด (Node) เป็นกลุ่มข้อมูลในโครงสร้างต้นไม้
 - กิ่ง (Branch) เป็นส่วนที่ใช้เชื่อมโหนดต่างๆ ในโครงสร้างต้นไม้

หาข้อมูลได้เร็วกว่า
การเก็บแบบ Linear List



Degree Node A = 3 (3EF)



TERMINOLOGY

Root : โหนดแรกสุดของต้นไม้

Parent : โหนดที่อยู่สูงกว่าโหนดที่พิจารณา 1 ระดับ

Child : โหนดที่อยู่ต่ำกว่าโหนดที่พิจารณา 1 ระดับ

Sibling : โหนดที่อยู่ระดับเดียวกันและมีพ่อเดียวกัน

Leaf : โหนดที่ไม่มีลูก

Ancestor : โหนดที่อยู่ในเส้นทางการเดินจากรูทมายังโหนดที่พิจารณา *บรรพบุรุษ*

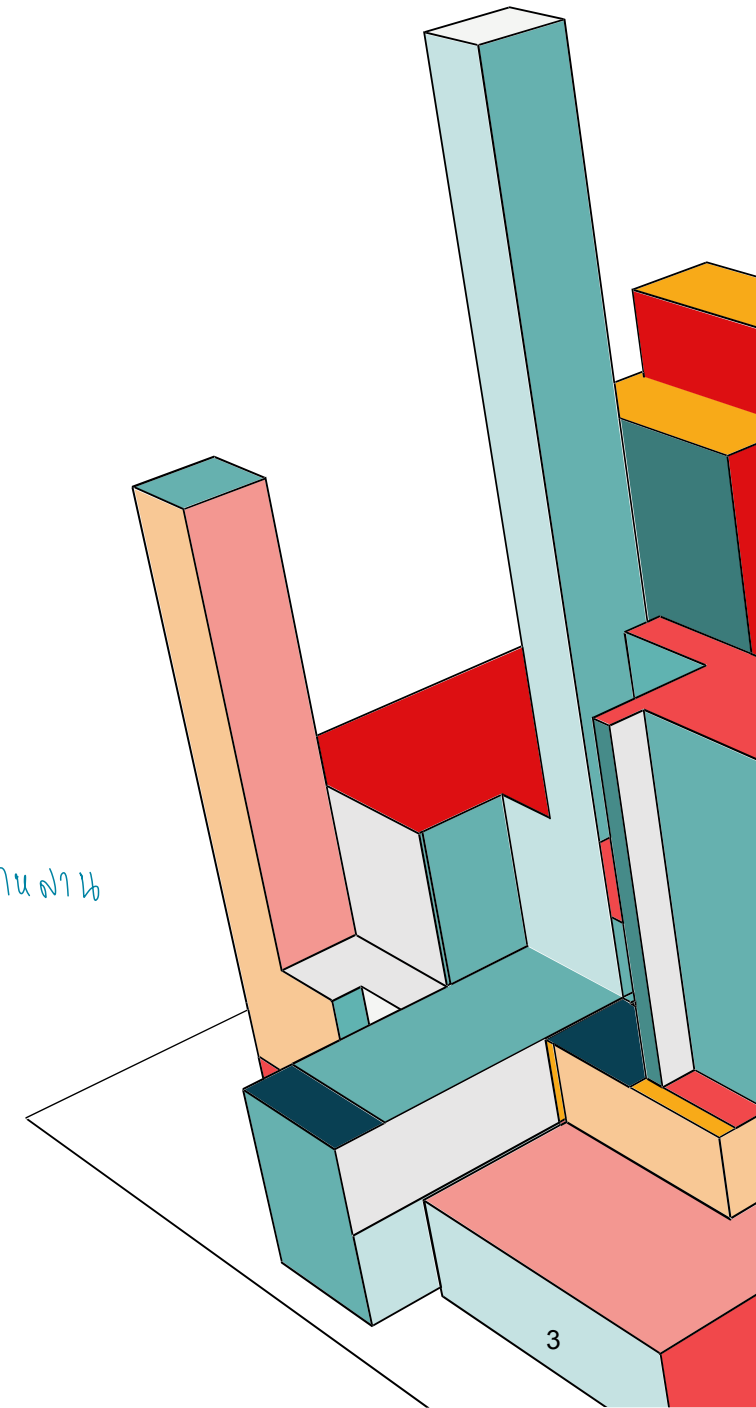
Descendent : โหนดที่อยู่ในเส้นทางการเดินจากโหนดที่พิจารณา ไปจนหมดต้นไม้ *ลูกหลาน*

Degree : จำนวนลูกของโหนดที่พิจารณา

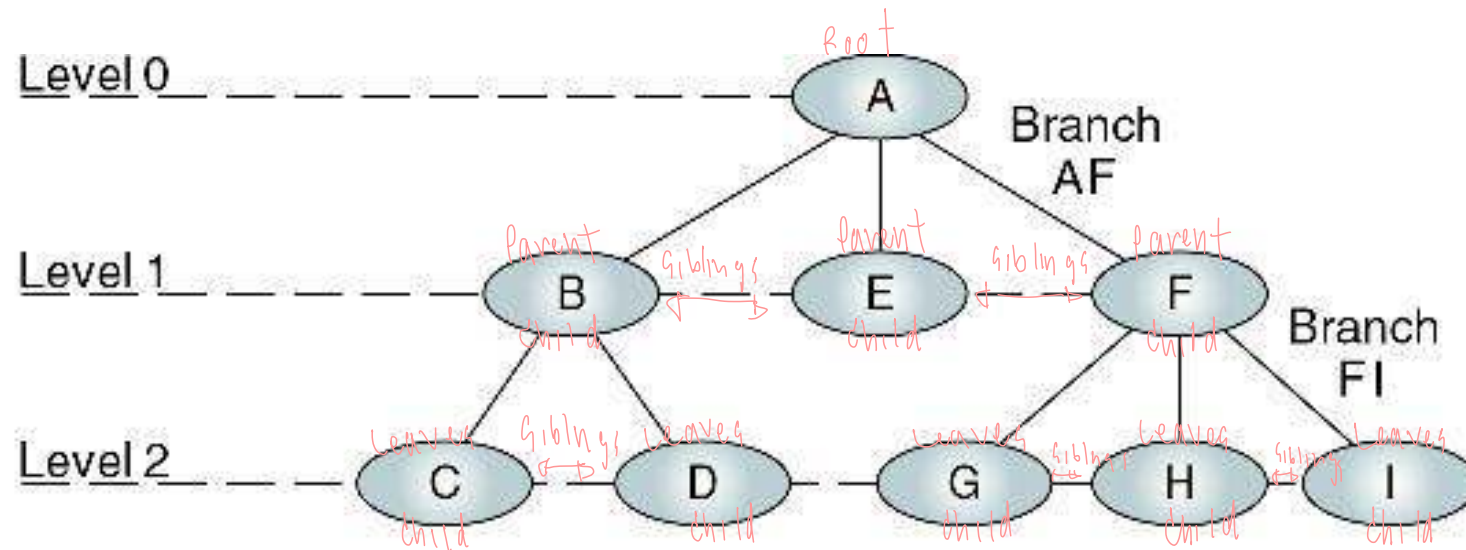
Level : ระยะทางจากรูทมายังโหนดที่พิจารณา

Depth : ความสูงของต้นไม้ (= Level ของโหนดที่อยู่ห่างจากรูทมากที่สุด + 1)

Subtree : ต้นไม้ย่อย



TERMINOLOGY (CONT.)



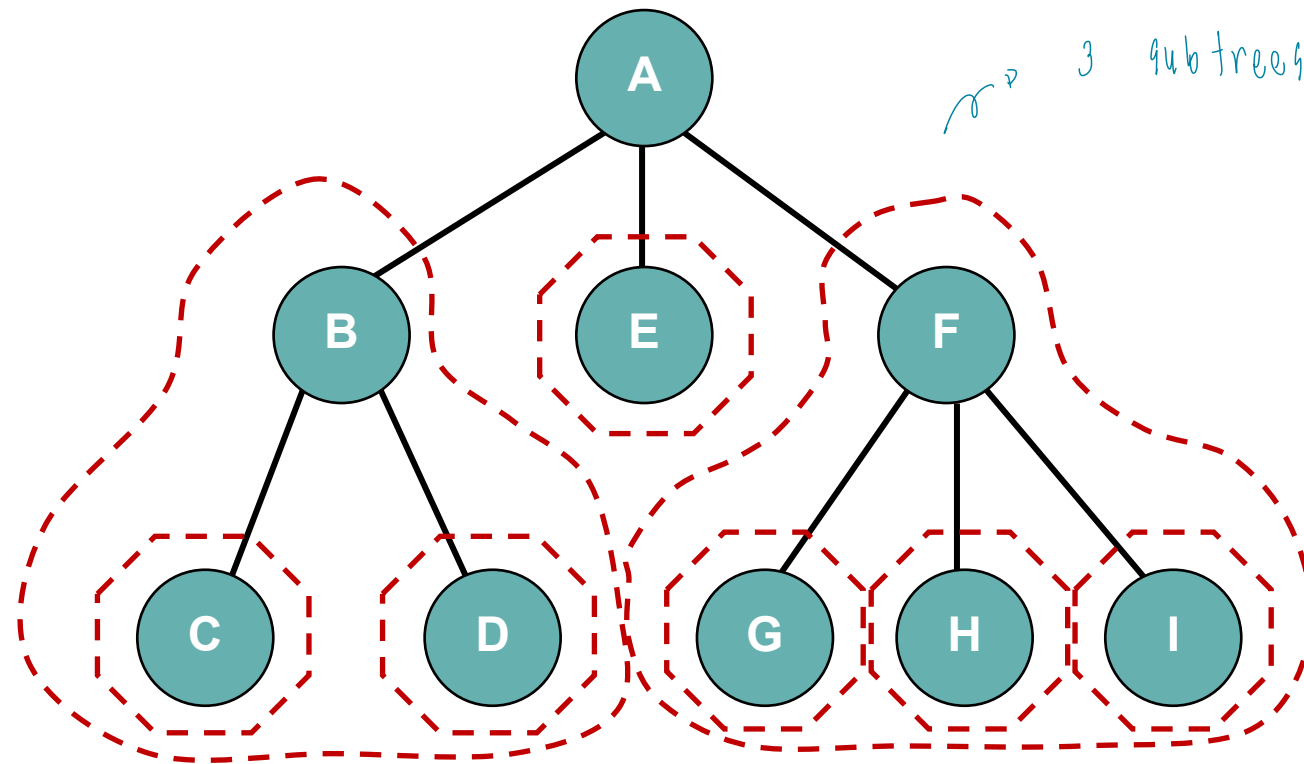
Root: A
Parents: A, B, F
Children: B, E, F, C, D, G, H, I

Siblings: {B, E, F}, {C, D}, {G, H, I}
Leaves: C, D, E, G, H, I
Internal nodes: B, F

↳ Node is 1st if it's root & 1st if it's leaves

SUBTREES

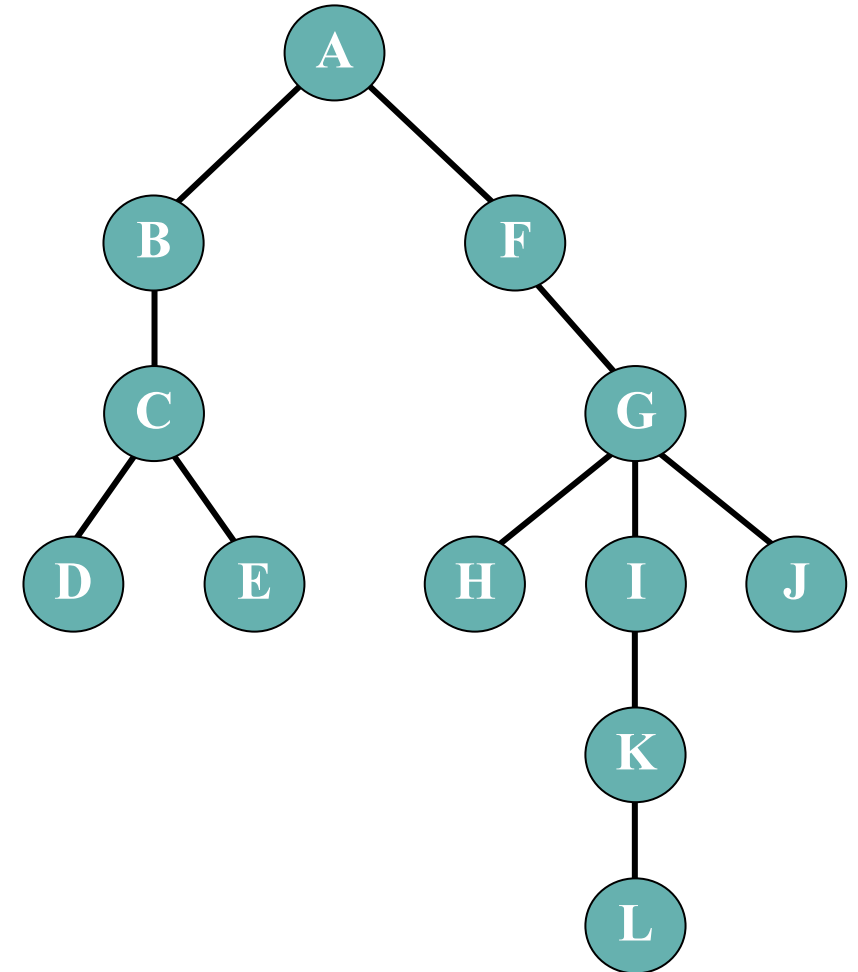
- โครงสร้างต้นไม้ย่อยๆ ในต้นไม้ใหญ่ทั้งต้น



TERMINOLOGY (CONT.)

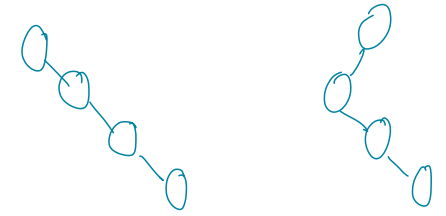
หาคำตอบต่อไปนี้

- Root : A
- Leaves : D E H L J
- Internal nodes : B C F G I K # Node ด้านใน
- Ancestors of H : A F G
- Descendents of F : G H I J K L
- Siblings of I : H J
- Degrees of L : 0
- Parent of K : I
- Children of C : D E
- Depth : 6
- Levels of the tree : 5
- Level of J : 3
- Number of all subtrees : 11



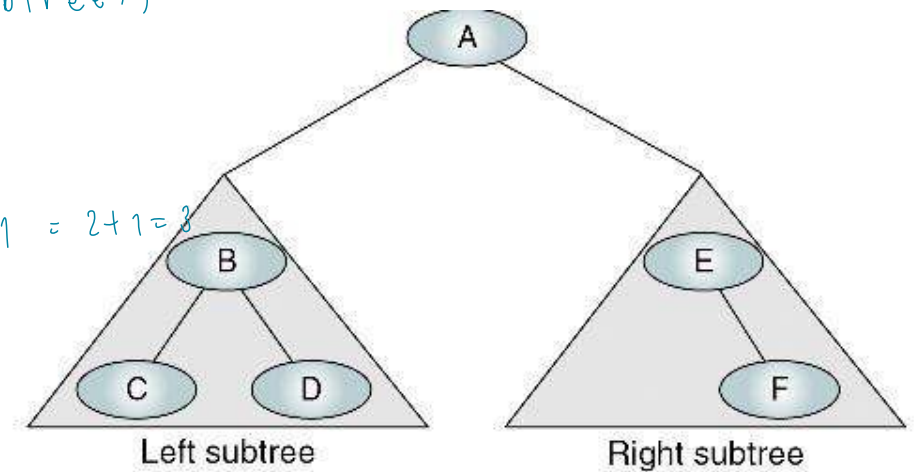
Ex. $N = 5 \rightarrow \log_2 5 + 1 = 3$ (ประมาณค่า)

$N = 4$



BINARY TREES

- เป็นโครงสร้างต้นไม้ ที่แต่ละโหนดจะมีโหนดลูกได้ไม่เกิน 2 ต้นไม้ย่อย
- ถ้าต้นไม้มีทั้งหมด N โหนด
 - Depth สูงสุด = N (เท่าจำนวน Node)
 - Depth น้อยสุด = $\log_2 N + 1$ (ทำไม้เต็มที่สุด) $\log_2 4 + 1 = 2 + 1 = 3$
- ถ้าต้นไม้มีความสูง D
 - จน.โหนดน้อยสุด = D
 - จน.โหนดมากที่สุด = $2^D - 1$



จำนวนโหนด = 2^{Level}
(เมื่อ Level)

$$\log_2^4 = 2 \quad \# \log_2 4 \text{ หาร 2}$$

$$\log_2^8 = 3$$

$$\log_2^{16} = 4$$

$$\log_3^{27} = 3 \quad \rightarrow \log_3^{\textcircled{3}} \xrightarrow{\text{คำตอบ}} 3$$

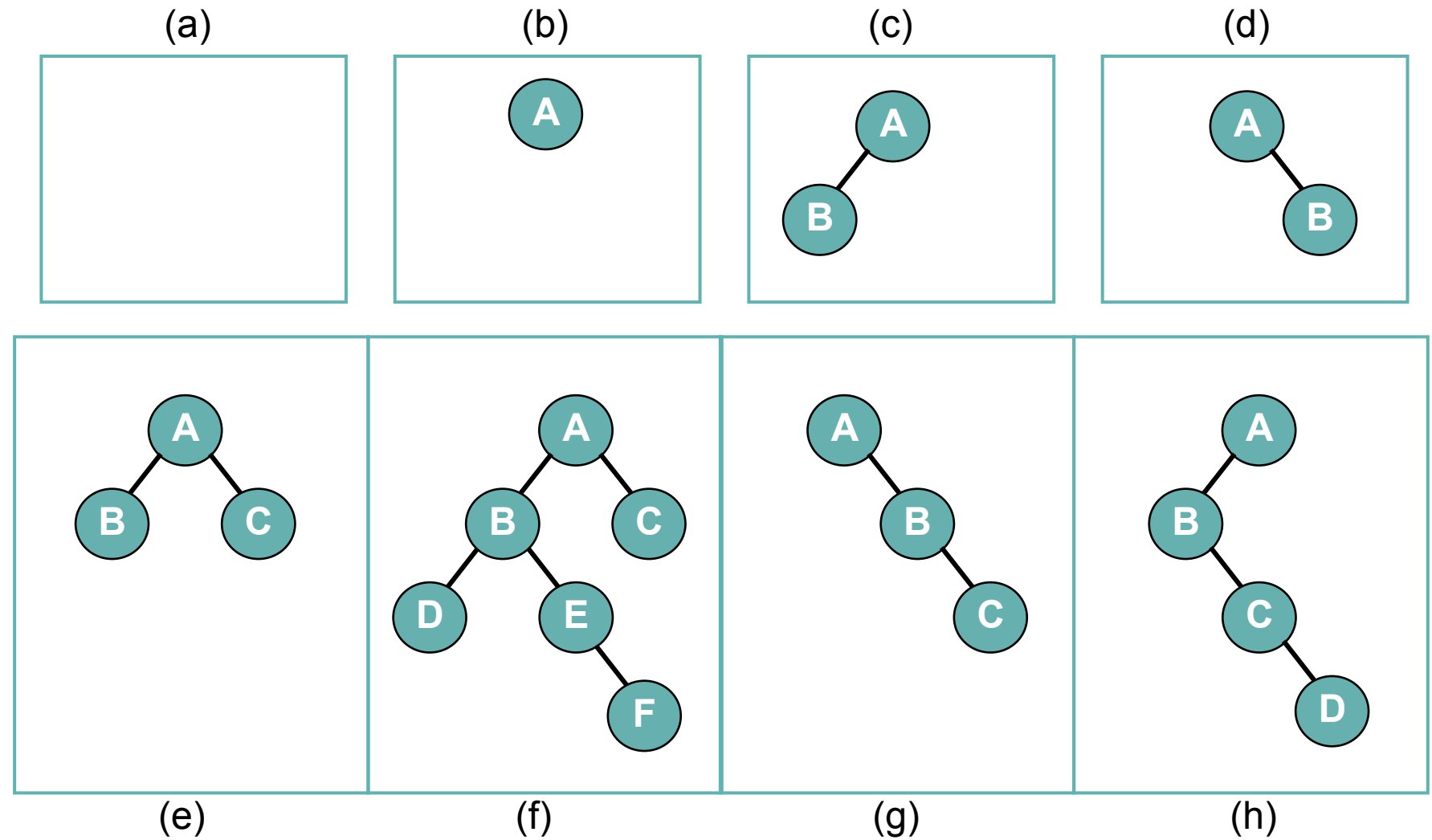
$$\log_2 6 = \log_2 4 < \log_2 6 < \log_2 8$$

$$2 \quad [2.58] \quad 3$$

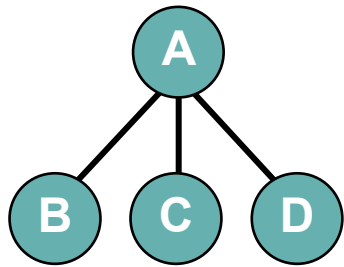
↳

ประมาณค่า

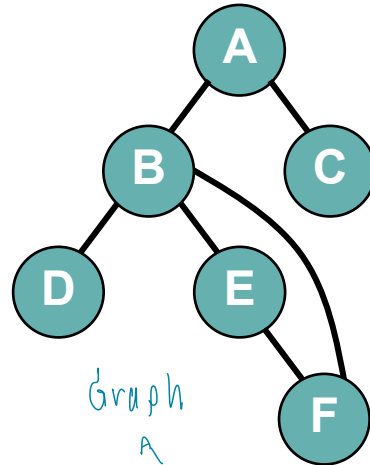
BINARY TREES



STRUCTURES WHICH ARE NOT BINARY TREES

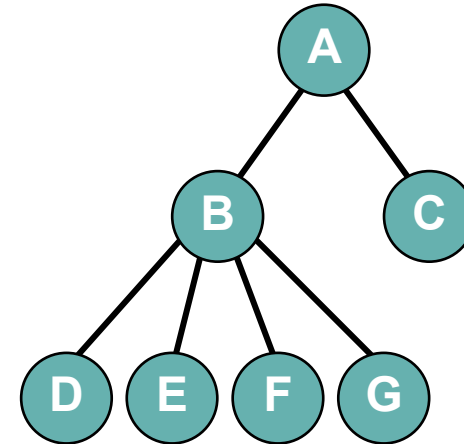


Tree



Graph
A

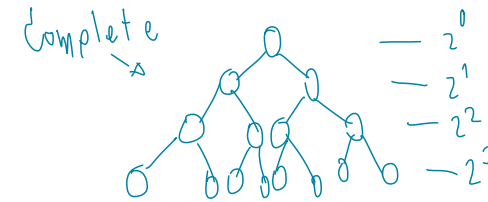
Not Tree!



Tree

COMPLETE AND

ต้นไม้ที่สมบูรณ์ !!



NEARLY COMPLETE TREES

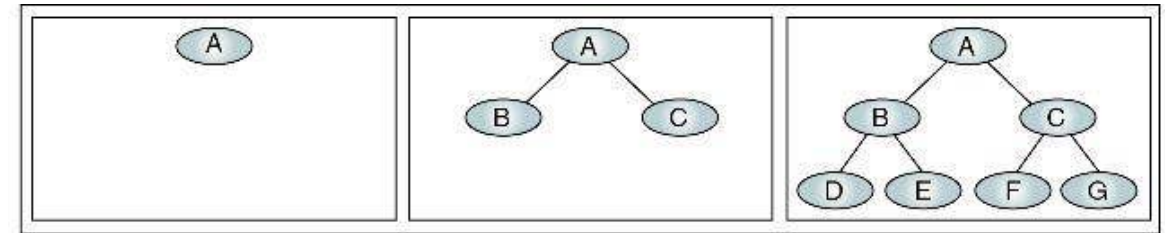
- **Complete Trees** : มีจนวน.โหนดสูงสุด เมื่อมี

ความสูง D

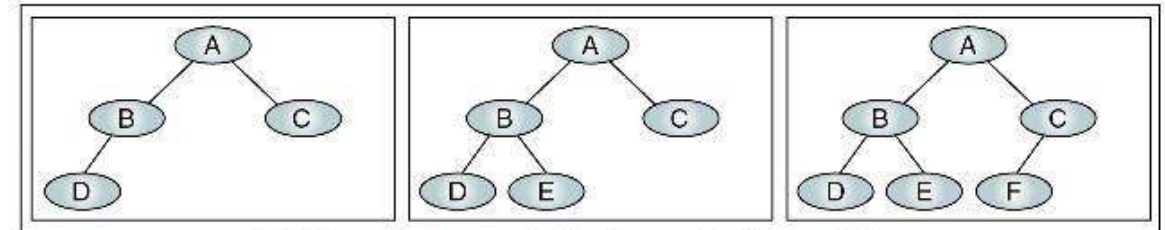
- $D = 3$
- $N = 2^3 - 1 = 7$

- **Nearly Complete Tree** : มีความสูงน้อยสุด
เมื่อมีโหนด N โหนด

- $N = 4, 5, 6$
- $D = \log_2 4 + 1 = 3$
- $D = \log_2 5 + 1 = 3$



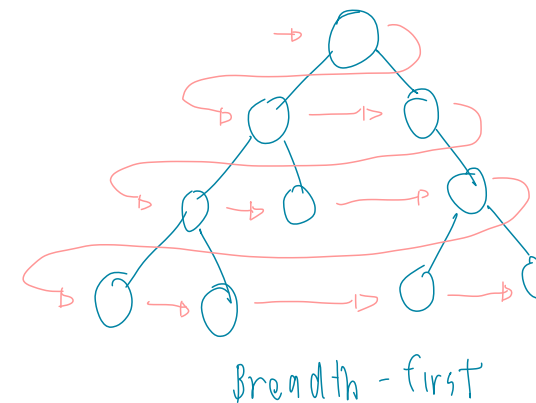
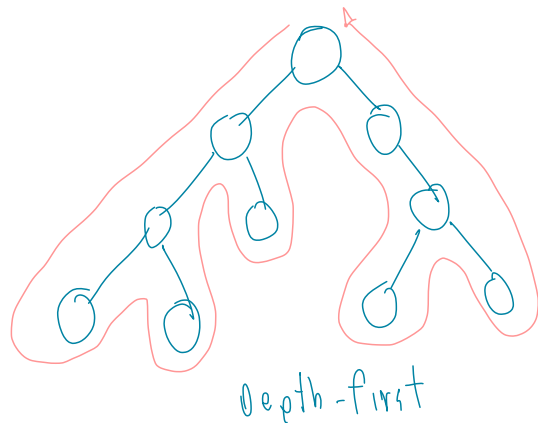
(a) Complete trees (at levels 0, 1, and 2)



(b) Nearly complete trees (at level 2)

BINARY TREE TRAVERSALS

- **Depth-first traversal** : เริ่มท่องจากรูท ไปใน descendent ของลูกตัวแรกจนหมด ค่อยไปท่องในลูกตัวถัด
- **Breadth-first traversal** : เริ่มท่องจากรูทแบบแนวกว้าง ไปยังลูกทุกตัวก่อน แล้วค่อยท่องไปที่ระดับลูกของลูกไปเรื่อยๆ

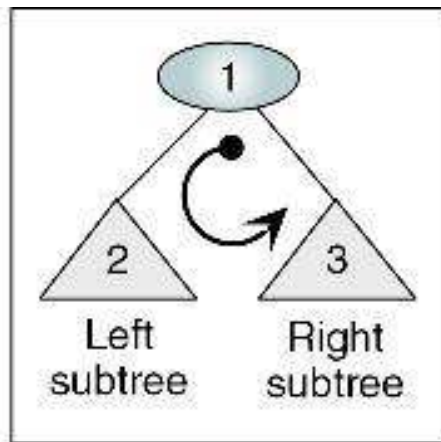


Prefix + AB
 Infix A + B
 Postfix AB +

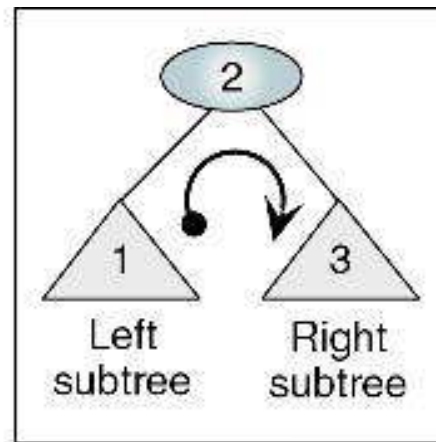
DEPTH-FIRST TRAVERSALS

- เริ่มท่องจากรูท ไปใน descendent ของลูกตัวแรกจนหมด ค่อยไปท่องในลูกตัวถัด

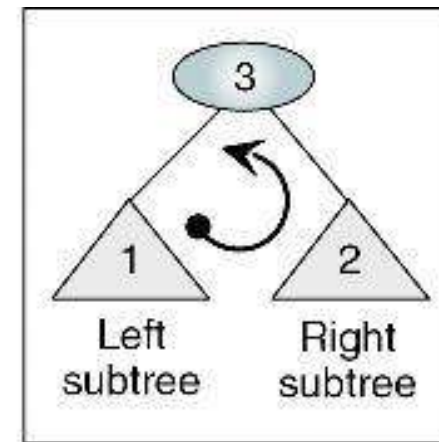
ทำ 166 หน้า ของ กาง
 ทำอีก



(a) **Preorder** traversal



(b) **Inorder** traversal



(c) **Postorder** traversal

PREORDER TRAVERSAL

Algorithm preOrder (root)

Traverse a binary tree in node-left-right sequence.

Pre root is the entry node of a tree or subtree

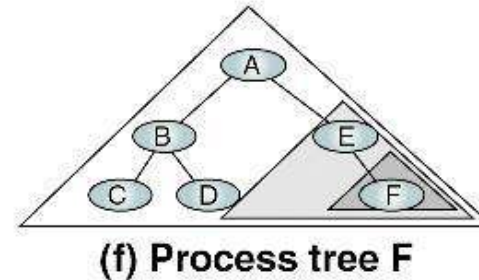
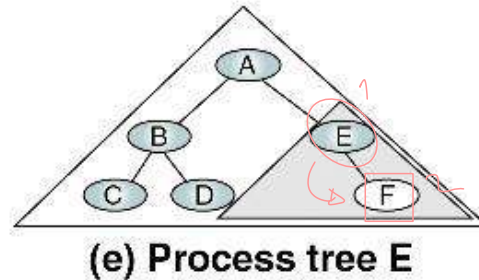
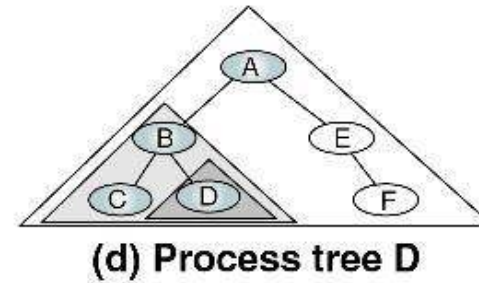
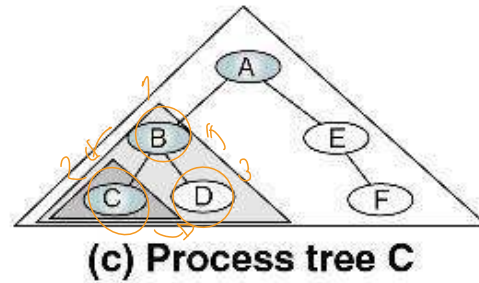
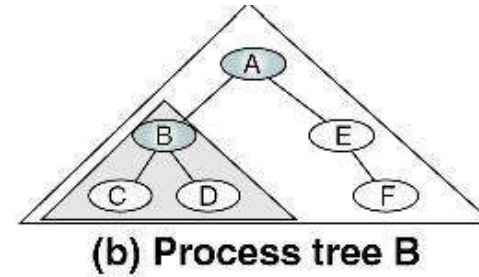
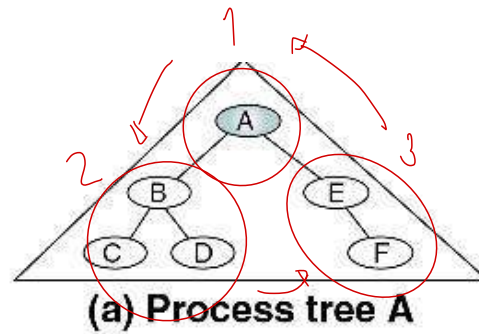
Post each node has been processed in order

```
1 if (root is not null)
    1 process (root)
    2 preOrder (leftSubtree)
    3 preOrder (rightSubtree)
2 end if
end preOrder
```

PREORDER TRAVERSAL (CONT.)

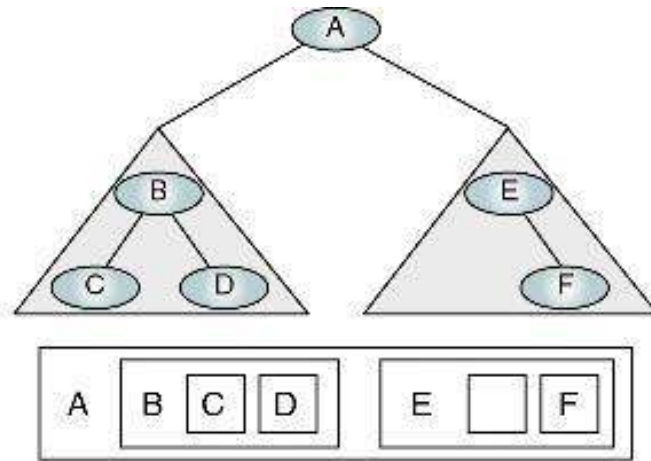
Root → Left → Right

A B C D E F

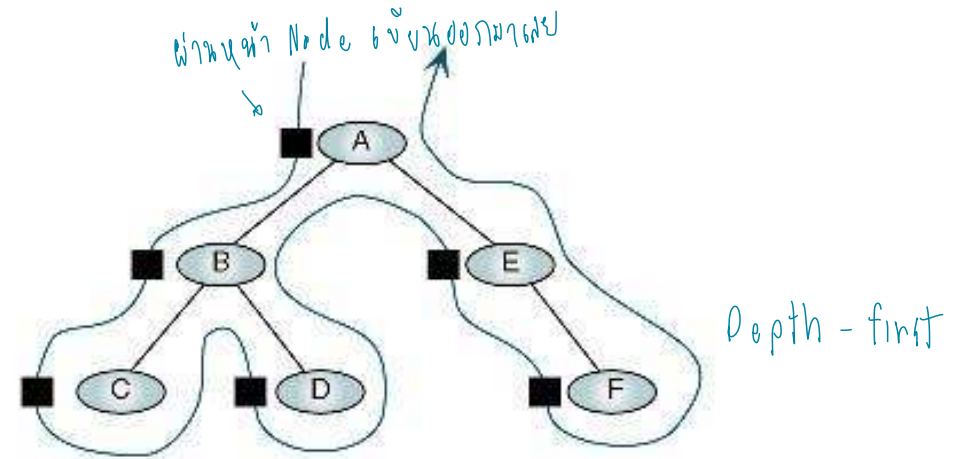


Algorithmic Traversal of Binary Tree

PREORDER TRAVERSAL (CONT.)



(a) Processing order



(b) "Walking" order

Preorder Traversal—A B C D E F

INORDER TRAVERSAL

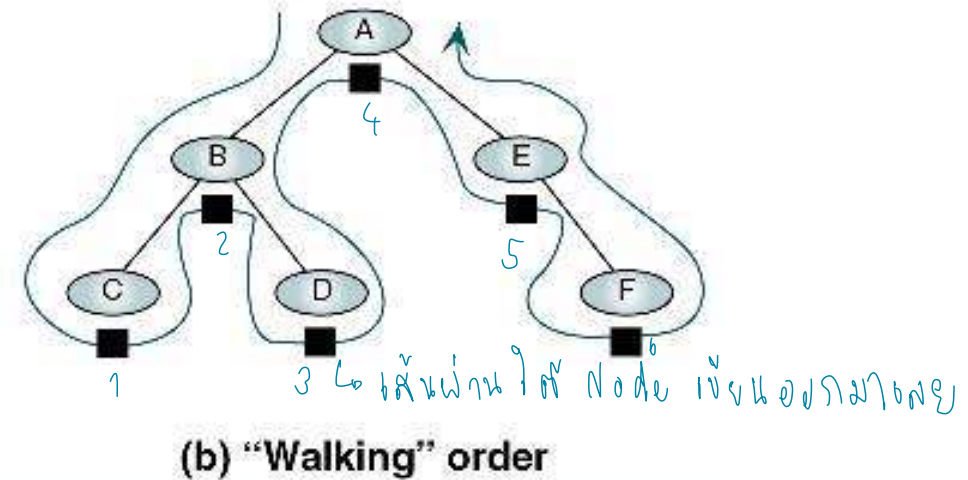
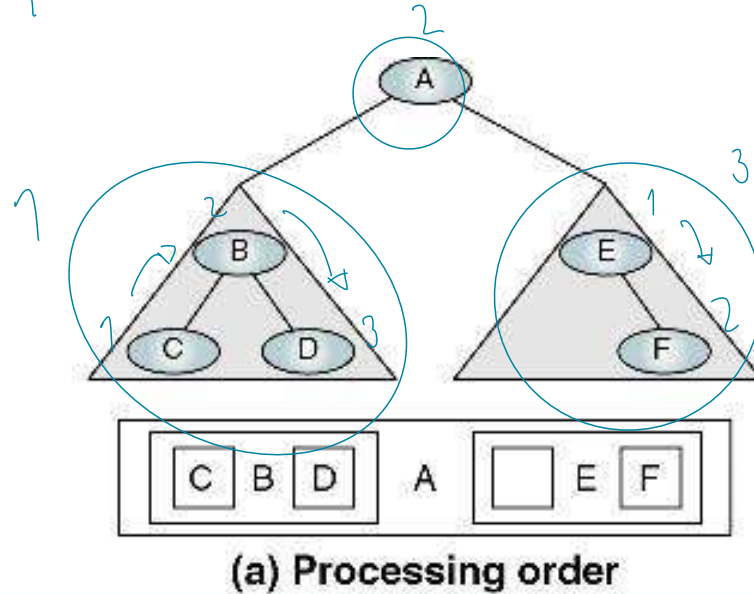
- ท่องตามลำดับ left subtree -> root -> right subtree

```
Algorithm inOrder (root)
  Traverse a binary tree in left-node-right sequence.
  Pre  root is the entry node of a tree or subtree
  Post each node has been processed in order
1  if (root is not null)
  1  inOrder (leftSubTree)
  2  process (root)
  3  inOrder (rightSubTree)
2  end if
end inOrder
```

INORDER TRAVERSAL (CONT.)

Left → Root → Right

A C B D E F



Inorder Traversal—C B D A E F

POSTORDER TRAVERSAL

- ท่องตามลำดับ left subtree -> right subtree -> root

```
Algorithm postOrder (root)
```

```
  Traverse a binary tree in left-right-node sequence.
```

```
    Pre  root is the entry node of a tree or subtree
```

```
    Post each node has been processed in order
```

```
1  if (root is not null)
```

```
    1  postOrder (left subtree)
```

```
    2  postOrder (right subtree)
```

```
    3  process (root)
```

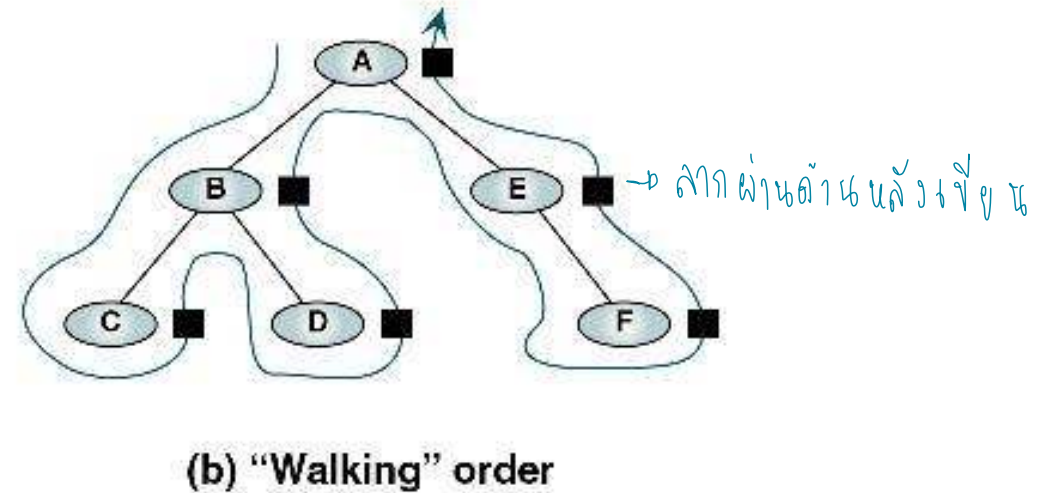
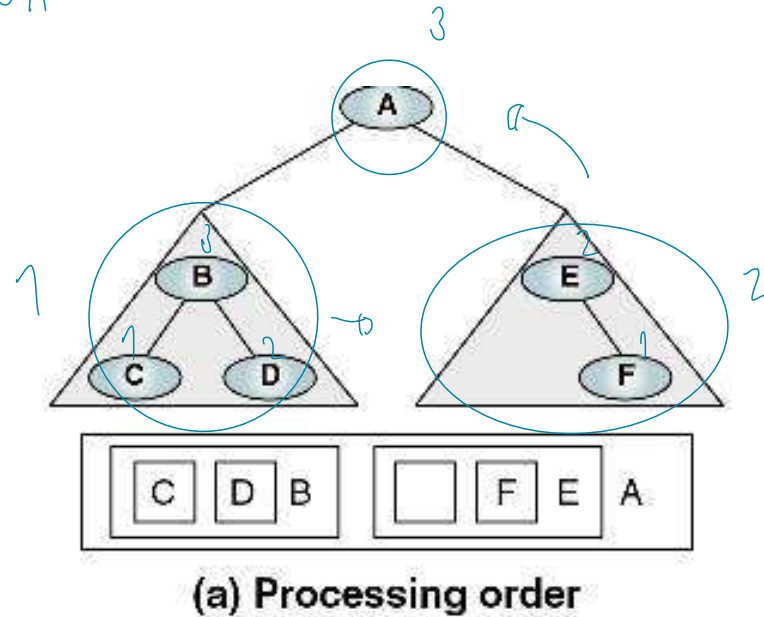
```
2  end if
```

```
end postOrder
```

POSTORDER TRAVERSAL

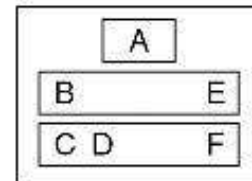
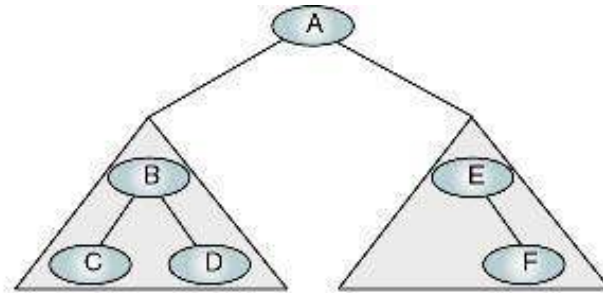
Left → Right → Root

C D B F E A

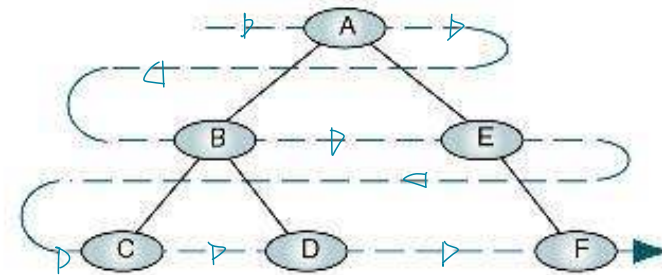


Postorder Traversal—C D B F E A

BREADTH-FIRST TREE



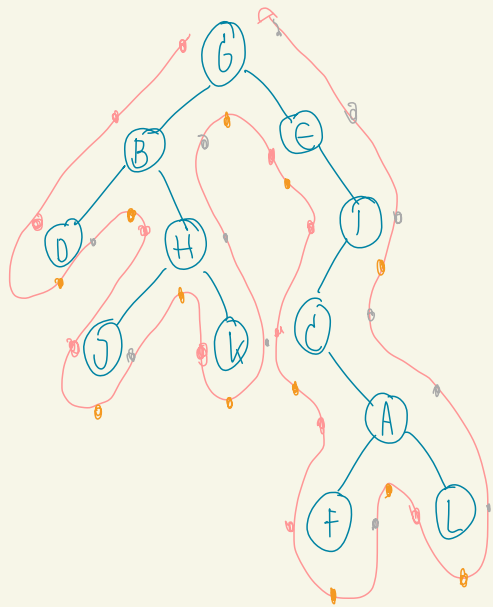
(a) Processing order



(b) "Walking" order

A B E C D F

Breadth-first Traversal

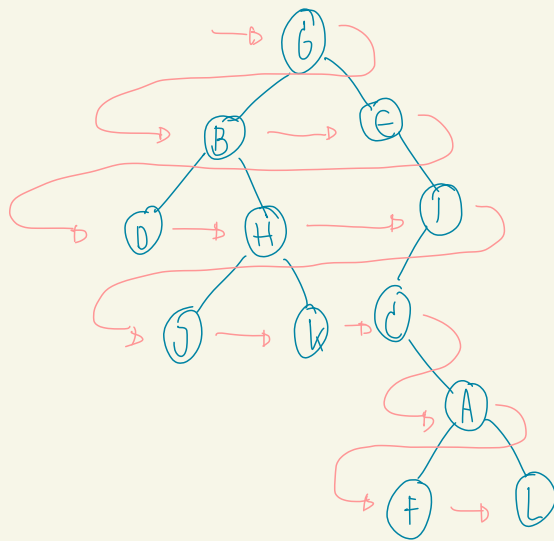


Depth first

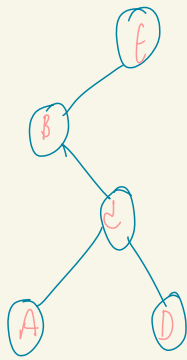
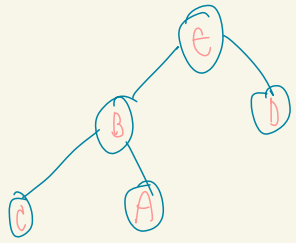
Preorder : ^{Root} G B D H J K E I C A F L (ऊपर)

Inorder : D B J H K ^{Root} G E C F A L I (संग)

Postorder : D J K H B F L A C I E ^{Root} G (नीचे)



Breadth-first : G B E D H I J K C A F L



Preorder : ^{Root} e B C A D



EXERCISE 1-4

- ❖ ให้หาารุทของ Binary Tree เมื่อมีลำดับการท่องเข้าไปในต้นไม้ดังนี้
 - a) Tree with postorder traversal: FCBDG
 - b) Tree with preorder traversal: IBCDFEN
 - c) Tree with inorder traversal: CBIDFGE
- ❖ ให้วาดรูป Binary Tree เมื่อมีลำดับการท่องเข้าไปในต้นไม้ดังนี้
Preorder : JCBADEFIGH
Inorder : ABCEDFJGIH
- ❖ ให้วาดรูป Binary Tree ที่เป็นไปได้ เมื่อมีโหนดทั้งหมด 3 โหนด (A,B,C) (บอกด้วยว่าเป็น complete, nearly complete หรือไม่)
- ❖ ให้วาดรูป Nearly complete binary tree ที่มีการท่องไปในต้นไม้แบบ Breadth-first traversal เป็น JCBADEFIGH